



Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Case Study: Class Weights on Bankruptcy

WEEK 2 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Two halves: the weights, then the case study

Hands-on, sections 10.1 and 10.2

`balanced_class_weight`: each class counts equally

What `class_weight` does inside `fit`

`output_vs_rate`: what the weights do to the output

Case study, sections 10.3 to 10.10

One setting at a time, by five folds

Class weights, none, or a lower threshold

Beside logistic regression and LightGBM

Paired intervals and a workbook

What it shows, and where it fails

326 of 4,680 training companies went bankrupt (6.97 percent). Do class weights help a network rank them? Promise nothing.

Class weights by hand

```
healthy weight = n / (2 * n0)      bankrupt weight = n / (2 * n1)
netkit.balanced_class_weight(y_fit) # equals compute_class_weight("balanced") within 1e-12
```

0.5375

healthy weight

7.1724

bankrupt weight

13.3448

times a healthy row

On the 3,744 fit rows the weights add up to 3,744.000000, the number of rows, and 0.500000 of the weight sits on bankrupt rows.

To the weighted loss the file looks half bankrupt: the weights pull every output up.

What class_weight does inside fit

1. Each row gets its class's weight (class_weight_to_sample_weights)
2. A batch's loss: sum of weight x log loss, divided by the rows

Weights	Keras	By hand	Keras divides by
balanced	0.213971	0.213971 (weighted mean)	rows = sum of weights
1 and 5	0.283352	0.283352 (rows), 0.221568 (sum)	the number of rows

Source read for the course: Keras 3.15.1, data_adapter_utils.py and losses/loss.py ("sum_over_batch_size").

What the weights do to the output

Stopping rows	Mean output	Bankrupt share	Ratio	Avg precision
no weights	0.0682	0.0694	0.9826	0.7331
balanced weights	0.1919	0.0694	2.7631	0.7480

The same network on 112 inputs (64 columns and 48 gap indicators), fitted twice on the 3,744 fit rows, stopped on the 936 stopping rows (65 bankrupt).

Allowed names

- model probability of bankruptcy in this file
- flagged at threshold t
- the model's score (class-weighted)

Never called

- a credit view on any company
- a rating
- an investment decision

The case study: context, objective, rows

Objective

- 1. Prepare as Course 4 did
- 2. One setting at a time, by five folds
- 3. Weights, none, or a lower threshold
- 4. Same folds for the earlier models
- 5. Paired intervals and a workbook
- 6. What it shows, and where it fails

Fold	Companies	Bankrupt
fold 1	936	65
fold 2	936	65
fold 3	936	65
fold 4	936	65
fold 5	936	66
test	closed	never scored

X1 net profit / total assets; X2 total liabilities / total assets (DATA.md). Attr1 is X1, Attr2 is X2, by position.

Model building by five folds

Kept only if the fold-mean average precision rises and the change is ahead on at least 4 of the 5 folds.

Model	Units	Dropout	Fold mean AP	Fold sd	Pooled AP
Model 1	(16, 8)	0.0	0.7284	0.0300	0.7206
Model 2	(16, 8)	0.2	0.6946	0.0604	0.6899

Model 2 ahead on 2 of 5 folds: not kept. Model 1 stands: 16 and 8 units, no dropout.

Class weights against a lower threshold

Out of fold	Threshold	Recall	Precision	Flagged
network at 0.5	0.5000	0.5276	0.8557	201
network at recall 0.70	0.1741	0.7025	0.5797	395
weighted network at 0.5	0.5000	0.7454	0.4508	539
weighted network at recall 0.70	0.6073	0.7025	0.5078	451

Pooled average precision: 0.7206 without weights, 0.6732 with them. The weighted network's mean output is 0.1583: a score.

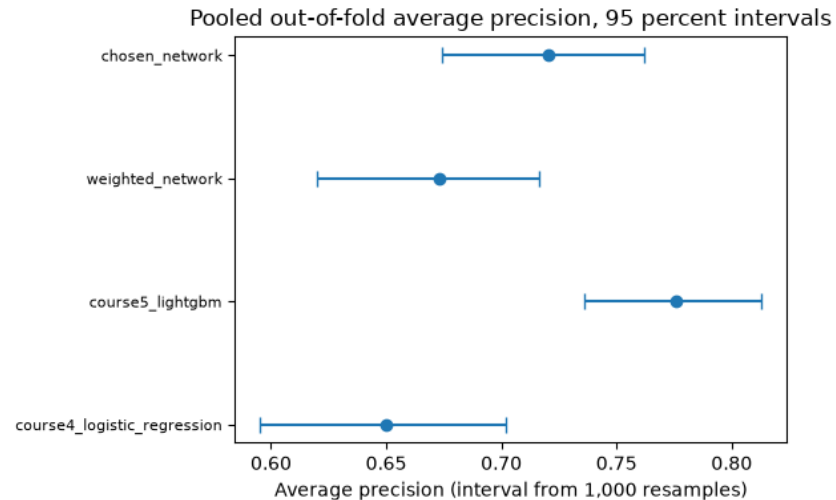
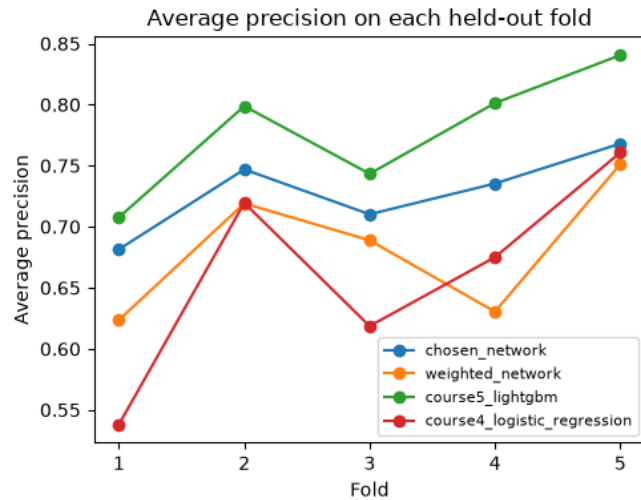
Decision: no class weights. A threshold chosen on out-of-fold outputs reaches recall 0.70 at a higher precision.

The comparison, by the same folds

Model	Pooled AP	Fold mean AP	Precision at 0.70	Flagged
network	0.7206	0.7284	0.5797	395
weighted network	0.6732	0.6826	0.5078	451
LightGBM	0.7759	0.7783	0.7003	327
logistic regression	0.6497	0.6624	0.4761	481

Fold means reproduce the earlier courses: LightGBM 0.7783, logistic regression 0.6624. Same rows, same folds.

Intervals on the pooled average precision



Network minus	Difference	Lower	Upper
weighted network	0.0474	0.0248	0.0709
LightGBM	-0.0553	-0.0837	-0.0296
logistic regression	0.0709	0.0377	0.1075

Separately the network's interval (0.6739 to 0.7619) overlaps LightGBM's (0.7359 to 0.8129). Paired, LightGBM and the logistic regression are clear of zero; the weighted network is too.

What the model shows, and where it fails

At the network's threshold	Attr1 median	Attr2 median	Companies
bankrupt, missed	0.0203	0.5564	97
bankrupt, flagged	-0.1195	0.7634	229
healthy, flagged	-0.0588	0.7156	166
healthy, not flagged	0.0549	0.4295	4188

Attr1: net profit / total assets. Attr2: total liabilities / total assets. Missed bankruptcies look sound on paper; flagged healthy companies show a loss.

63 of the 97 bankruptcies the network misses are missed by all three models. Outputs correlate 0.8759 with LightGBM's.

Excel side by side

Excel side by side (tested in Excel)

```
=ROWS(A2:A3745)/(2*COUNTIF(A2:A3745,1))  
=IF(A2=1,$F$2,$F$1)  
=SUMPRODUCT(C2:C3745,D2:D3745)/SUM(C2:C3745)  
=SUM(C2:C3745)  
=SUMPRODUCT(C2:C3745,A2:A3745)/SUM(C2:C3745)  
=COUNTIFS(A2:A4681,1,C2:C4681,1)  
=PERCENTILE.INC(D2:D1001,0.025) and 0.975
```

bankrupt weight: 7.1724
each row's weight, in C2, filled down
weighted mean log loss: 0.213971
the number of rows: 3,744
share on bankrupt rows: 0.5000
bankrupt flagged: 229; precision 0.5797
minus LightGBM -0.0837 to -0.0296

No network training, class weights in a fit, cross-validation, or seeded bootstrap in Excel. Every number equals Python's.

Save the week: week1 and week2

1. Week 1

netkit cut back; git tag week1

2. Week 2

today's netkit, byte for byte

3. Log

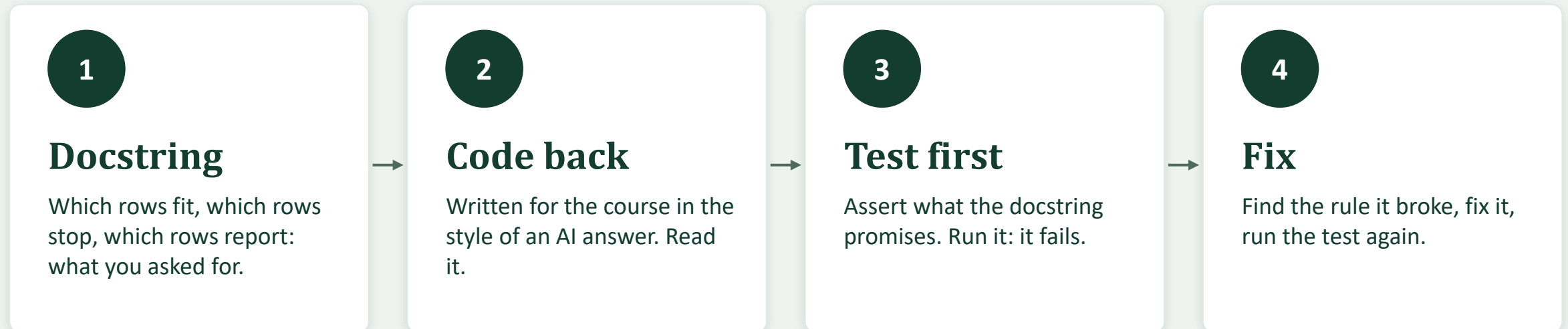
four cv rows; git tag week2

```
git diff --stat week1 week2
netkit.py      | 235 ++++++++++++++++++++++++++++++++++++++
results.csv    |   5 ++
test_netkit.py | 210 ++++++++++++++++++++++++++++++++++++++
3 files changed, 450 insertions(+)
```

43 tests pass. mlkit.py and ensemblekit.py are not in the list: week 2 never changed them.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.



Today: the keys are swapped, so the bankrupt rows carry 0.0056 of the weight instead of half.

Code that runs is not the same as code that is right.

What you can do now

- Work out balanced class weights, and why they add to n
- Say what `class_weight` does inside `fit`
- Show a weighted network's output is a score
- Weights against a lower threshold, out of fold
- Paired intervals; what it shows and where it fails

Week 3, Day 11 Windows and walk-forward on the yield curve